

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
FACULTAD DE INGENIERÍA • DEPARTAMENTO DE SISTEMAS
BASES DE DATOS I

INFORME TÉCNICO DE AULA RTF4
IMPLEMENTACIÓN Y REFINAMIENTO DEL MODELO
LÓGICO DDL

Motor Relacional de Gestión Económica, Trazabilidad Contable e Inmutabilidad Transaccional
para Servidores de Rol GTA (Economy RP)

Proyecto de Aula:	Economy RP (Motor Económico Relacional)
Equipo de Trabajo:	Grupo 4
Docente Asesor:	Juan Camilo Palacio Alcaraz
Integrantes:	• Miguel Ángel Cardona Agudelo • Luisa María López Orrego • Jorge Luis Ordoñez Ávila
Materia / Semestre:	Bases de Datos I • Semestre 2026-2
Fecha de Entrega:	03 de Octubre de 2026

RÚBRICA DE EVALUACIÓN OFICIAL RTF4 (100% PONDERADO)	PESO
1. Evolución y Refinamiento del Modelo Lógico (Antes vs. Después)	25% (1.25)
2. Definición y Estructura del Esquema DDL Tabular	25% (1.25)
3. Informe de Mejoras y Proceso de Normalización (1FN, 2FN, 3FN, BCNF, 4FN, DKNF)	20% (1.00)
4. Pruebas Estructurales, Hallazgos y Manejo de Errores de Motor SGBD	20% (1.00)
5. Estructura, Formato Institucional y Defensa Técnica del Modelo	10% (0.50)

1. Objetivo de la Entrega

El objetivo principal de esta fase es la materialización y refinamiento del modelo lógico relacional en el Sistema Gestor de Bases de Datos (MySQL 8.0 con motor transaccional InnoDB), a partir del diseño conceptual previo. Se busca implementar y certificar la robustez del esquema DDL físico para validar la integridad referencial, suprimir inconsistencias y asegurar el estricto cumplimiento de las reglas del negocio de rol, resolviendo fallas críticas tales como:

- **Inconsistencias en tipos de datos:** Estandarización a DECIMAL(12,2) para evitar pérdidas de precisión por coma flotante.
- **Definición rigurosa de claves primarias y foráneas:** Eliminación de claves compuestas ambiguas y soporte de llaves subrogadas.
- **Restricciones de integridad y dominio (CHECKs):** Blindaje de estados y tipos contables para evitar datos huérfanos o no tipificados.
- **Consistencia transaccional y atomicidad (ACID):** Asegurar que en negociaciones de intercambio P2P y liquidación laboral ningún bien ni saldo quede duplicado o en el limbo.
- **Inmutabilidad en el libro mayor contable:** Estructuración de un historial de transacciones *append-only* con partida doble.
- **Automatización de políticas económicas:** Control en tiempo real de límites de posesión por jugador, periodos de enfriamiento laboral, tasas de drenaje anti-inflacionario y detección reactiva de bancarrota.

2. Catálogo Oficial de Reglas de Negocio (RN001 - RN012)

A continuación se detallan las reglas de negocio formalizadas para el dominio económico del servidor de rol GTA (Economy RP), expresando su condición lógica y su mecanismo de implementación exclusivo en la base de datos:

CÓD.	NOMBRE	DESCRIPCIÓN FUNCIONAL	FÓRMULA / CONDICIÓN	IMPACTO DDL / SGBD
RN001	Registro Único Laboral	Al completar una jornada, el sistema calcula y acredita automáticamente la ganancia a la cuenta del jugador.	$Ganancia = Tarifa_base \times Horas$	Tabla jornadas_laborales, PK subrogada id_jornada y columna histórica inmutable monto_pagado.
RN002	Propiedad Única y Exclusiva	Un bien físico (vehículo con placa, propiedad) solo puede pertenecer a un único personaje a la vez.	Propietario = 1 por cada Bien activo	Clave foránea atómica items.id_jugador con integridad referencial directa (1:N estricto).
RN003	Inmutabilidad del Historial	Toda transacción financiera o comercial registrada no puede ser editada ni eliminada bajo ninguna circunstancia.	Política Append-Only (Sin UPDATE / DELETE)	Tablas transacciones y detalles_transacciones inmutables protegidas por disparadores.
RN004	Restricción por Deuda / Embargo	Un bien no puede transferirse si posee deuda activa, embargo o cuotas financieras pendientes.	Restriccion = 'Ninguna' (tiene_deuda = FALSE)	Columna tiene_deuda BOOLEAN NOT NULL DEFAULT FALSE y validación en procedimiento de tradeo.
RN005	Trazabilidad de Creación de Dinero	Todo dinero inyectado (salarios, bonos iniciales) debe originarse formalmente de la cuenta oficial 'SISTEMA'.	Origen = Cuenta_Sistema si Emisor = Sistema	cuentas.tipo_cuenta = 'SISTEMA' (id_jugador NULL) y asiento de partida doble en detalles.
RN006	Comisión por Transferencia	Toda compraventa o intercambio descuenta una comisión porcentual que se acredita a la Tesorería como drenaje anti-inflación.	$Comisión = Monto \times Porcentaje_Comisión$	Atributo servidores.porcentaje_comision y asiento secundario de drenaje fiscal en detalles.
RN007	Límite Máximo de Propiedades	Un jugador no puede registrar más bienes que el tope máximo parametrizado por el administrador.	$COUNT(Items) \leq Limite_Bienes_Servidor$	Atributo servidores.limite_bienes_por_jugador y control de aforo en inserción/tradeo.
RN008	Enfriamiento Laboral (Cooldown)	Un jugador debe cumplir un tiempo mínimo de descanso entre turnos del mismo empleo para evitar explotación.	$Hora_Actual - Hora_Ultima \geq Tiempo_Cooldown$	Atributo servidores.tiempo_enfriamiento_min e inspección temporal en inserción laboral.
RN009	Penalización por Bancarrota	Si el saldo disponible no cubre obligaciones o queda en negativo, el jugador pasa a estado 'BANCARROTA', bloqueando operaciones.	$Saldo < 0 \Rightarrow Estado = 'BANCARROTA'$	Restricción CHECK (estado IN ('ACTIVO', 'BANCARROTA', 'SUSPENDIDO')) en jugadores.
RN010	Confirmación Doble en Tradeo	Ambas partes deben confirmar explícitamente los términos. Cualquier modificación reinicia los candados de confirmación.	Ejecutar si (conf_j1 = TRUE AND conf_j2 = TRUE)	Banderas booleanas confirmacion_j1 y confirmacion_j2 en negociaciones_tradeos.
RN011	Límite Diario de Tradeos	Un operador no puede ejecutar más intercambios exitosos al día que el cupo configurado, evitando lavado de activos.	$COUNT(Tradeos_Hoy) \leq limite_tradeos_diarios$	Columna servidores.limite_tradeos_diarios y agregación temporal sobre DATE(fecha_hora).

RN012	Tradeo Único Simultáneo	Un jugador solo puede tener una negociación abierta a la vez, impidiendo comprometer saldos o ítems en paralelo.	COUNT(Negociaciones_Activas) <= 1	Restricción de dominio sobre el ciclo de vida CHECK (estado IN (...)) en negociaciones.
--------------	--------------------------------	--	-----------------------------------	---

3. Evolución del Modelo Lógico (Antes vs. Después)

3.1 Diagnóstico de Fallas en el Modelo Lógico Inicial (Antes)

En la fase conceptual y lógica preliminar (RTF3), el modelo presentaba severas limitaciones estructurales que impedían su ejecución eficiente en un SGBD relacional de producción. Entre las fallas identificadas se destacan:

- **Violación de 1FN por atributos multivaluados:** En Transaccion se pretendía registrar cadenas de texto como Usuarios_Involucrados = 'Emisor: Carlos, Receptor: Ana' y Bienes_Afectados = 'Vehículo, Propiedad', imposibilitando búsquedas relacionales o integridad referencial.
- **Clave compuesta ambigua en jornadas (2FN):** Se intentó definir (id_jugador, id_empleo) como PK compuesta, bloqueando a los jugadores de realizar más de un turno laboral en el mismo oficio a lo largo del tiempo (Error 1062 Duplicate Key).
- **Restricción de unicidad rígida en cuentas (1:1):** En T_Cuenta se definía id_jugador UNIQUE, impidiendo que un operador administrara cuentas de ahorro, corrientes o comerciales, además de bloquear la cuenta central de Tesorería por falta de un titular físico.
- **Falta de atomicidad en permutas:** El tradeo original solo permitía transferencias unilaterales (1 ítem por dinero), sin soporte para intercambios cruzados simultáneos de activos y fondos.
- **Nomenclatura no normalizada:** Uso redundante del prefijo T_ y nombres en singular que dificultaban el estándar relacional.

3.2 Modelo Lógico Corregido y Refinado (Después)

Tras la implementación en el SGBD MySQL (InnoDB), el esquema DDL se consolidó resolviendo las fallas conceptuales previas mediante las siguientes transformaciones estructurales:

ENTIDAD	DISEÑO INICIAL (RTF3 / ANTES)	DISEÑO CORREGIDO (RTF4 / DESPUÉS)	MEJORA TÉCNICA APORTADA
servidores	Contenedor conceptual sin atributos claros de gobernanza.	Tabla con porcentaje_comision, limite_bienes, cooldown_min y limite_tradeos_diarios.	Centraliza la parametrización de RN006, RN007, RN008 y RN011.
jugadores	Atributos genéricos, credenciales en texto plano, sin restricción de estado.	PK simple, FK a servidores, username/correo únicos, hash criptográfico y CHECK (estado IN (...)).	Integridad referencial, seguridad de acceso y soporte formal de estados judiciales (RN009).
empleos	Mezclaba horas trabajadas con la definición del oficio.	Catálogo desacoplado con id_empleo PK, FK a servidor, nombre y tarifa_base DECIMAL(10,2).	Desacoplamiento total entre la oferta laboral y los turnos individuales ejecutados.
jornadas_laborales	Planteada con PK compuesta (jugador + empleo) y ganancia calculada dinámica.	PK subrogada id_jornada, FKs independientes, horas trabajadas y monto_pagado inmutable.	Cumplimiento estricto de 2FN y 3FN; historial laboral auditable sin desincronización de tarifas.
cuentas	Atada 1:1 rígidamente a Jugador con UNIQUE. Sin cuenta oficial del sistema.	PK simple, FK a jugador (nullable), FK a servidor, tipo_cuenta CHECK ('PERSONAL','SISTEMA').	Habilita la Tesorería Central (RN005) y elimina la restricción que impedía cuentas múltiples (1:N).
items	Categoría como texto libre, fechas redundantes y sin control de deudas.	PK simple, FK a jugador, precio tasado, fecha adquisición, flag tiene_deuda y estado de custodia.	Cumple RN002 (un dueño por bien) y RN004 (bloqueo por gravamen o embargo).
negociaciones_tradeos	Inexistente; se pretendía ejecutar el intercambio directamente sobre transacciones.	Mesa bilateral simétrica con id_item_j1/j2, monto_j1/j2, doble confirmación y expiración.	Garantiza la máquina de estados de negociación (RN010, RN012) con soporte de permuta cruzada.
transacciones	Sobrecargada con listas de texto concatenadas (usuarios y bienes en un campo).	Cabecera inmutable de libro mayor con tipo de operación, monto nominal, fecha_hora y estado.	Estructura de auditoría contable de sólo adición (*append-only*) inalterable (RN003).
detalles_transacciones	Inexistente en el diseño preliminar.	Asientos contables por partida doble con cuenta_origen, cuenta_destino, tipo de movimiento y monto.	Trazabilidad contable estricta al centavo y cumplimiento de 1FN.

4. Estructura del Esquema Tabular (DDL)

A continuación se documenta de forma exhaustiva y rigurosa la estructura tabular de las entidades físicas en el motor MySQL (InnoDB), especificando tipos de datos, nulidad, claves, valores por defecto y su justificación técnica vinculada a las reglas de negocio:

Tabla: servidores (Configuración y Gobernanza Económica)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_servidor	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único de la instancia del servidor.
nombre	VARCHAR(100)	NO	N/A	N/A	Nombre institucional del servidor de rol.
porcentaje_comision	DECIMAL(5,2)	NO	CHECK (>= 0)	5.00	Tasa de drenaje fiscal en transacciones (RN006).
limite_bienes_por_jugador	INT	NO	CHECK (> 0)	100	Tope máximo de inventario por operador (RN007).
tiempo_enfriamiento_min	INT	NO	CHECK (>= 0)	15	Minutos obligatorios de descanso laboral (RN008).
limite_tradeos_diarios	INT	NO	CHECK (> 0)	5	Cupo máximo de tradeos diarios por usuario (RN011).

Tabla: jugadores (Usuarios y Operadores del Servidor)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_jugador	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador correlativo del jugador.
id_servidor	INT	NO	FK → servidores(id)	1	Servidor al que pertenece el jugador.
nombre_usuario	VARCHAR(50)	NO	UNIQUE	N/A	Tag de acceso y apodo único del personaje.
correo	VARCHAR(150)	NO	UNIQUE	N/A	Dirección de correo electrónico personal.
contrasena_hash	VARCHAR(255)	NO	N/A	N/A	Hash criptográfico seguro de credenciales.
fecha_registro	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Marca temporal de alta en el sistema.
estado	VARCHAR(20)	NO	CHECK ('ACTIVO','BANCAROTA','SUSPENDIDO')	'ACTIVO'	Estado operativo y judicial del operador (RN009).
es_admin	BOOLEAN	NO	N/A	FALSE	Bandera de privilegios administrativos.

Tabla: empleos (Catálogo de Oficios y Tarifas)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_empleo	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único de la plaza laboral.
id_servidor	INT	NO	FK → servidores(id)	1	Servidor que habilita el empleo.
nombre_empleo	VARCHAR(100)	NO	N/A	N/A	Denominación del oficio (ej. Minero, Mecánico).
tarifa_base	DECIMAL(10,2)	NO	CHECK (> 0)	N/A	Remuneración base por unidad de trabajo (RN001).

Tabla: jornadas_laborales (Historial de Turnos de Trabajo)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_jornada	INT	NO	PK, AUTO_INCREMENT	N/A	Clave subrogada única del turno ejecutado.
id_jugador	INT	NO	FK → jugadores(id)	N/A	Operador que desempeñó la jornada laboral.
id_empleo	INT	NO	FK → empleos(id)	N/A	Oficio desempeñado durante el turno.
horas_trabajadas	DECIMAL(6,2)	NO	CHECK (> 0)	N/A	Unidades de tiempo laboradas en la jornada.
fecha_hora	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Cierre de turno registrado por defecto (RN008).
monto_pagado	DECIMAL(10,2)	NO	CHECK (>= 0)	0.00	Monto histórico liquidado inmutable (RN001, 3FN).

Tabla: cuentas (Fondos Bancarios y Tesorería Central)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_cuenta	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único de la cuenta monetaria.
id_jugador	INT	SÍ	FK → jugadores(id)	NULL	Titular. NULL si representa la Tesorería (RN005).
id_servidor	INT	SÍ	FK → servidores(id)	NULL	Servidor al que pertenece la cuenta bancaria.
tipo_cuenta	VARCHAR(20)	NO	CHECK ('PERSONAL','SISTEMA')	'PERSONAL'	Tipo de cuenta contable (RN005).
saldo_inicial	DECIMAL(12,2)	NO	N/A	0.00	Monto de apertura registrado en el sistema.
saldo_disponible	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Balance líquido activo para transacciones (RN009).

Tabla: items (Inventario de Activos y Bienes)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_item	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único de activo/propiedad.
id_jugador	INT	NO	FK → jugadores(id)	N/A	Propietario legítimo único del bien (RN002).
nombre	VARCHAR(100)	NO	N/A	N/A	Nombre comercial, modelo o matrícula del bien.
precio	DECIMAL(12,2)	NO	CHECK (>= 0)	N/A	Valor de avalúo comercial tasado.
fecha	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Fecha y hora de adquisición o inyección del bien.
tiene_deuda	BOOLEAN	NO	N/A	FALSE	Indicador de gravamen o deuda pendiente (RN004).
antiguedad_dias	INT	NO	CHECK (>= 0)	0	Días acumulados para cálculo dinámico de depreciación.
estado_custodia	VARCHAR(20)	NO	CHECK ('PERSONAL','EMBARGADO','EN_TRADE')	'PERSONAL'	Estado de posesión física y bloqueo transaccional.

Tabla: negociaciones_tradeos (Mesa Bilateral de Intercambio P2P)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_negociacion	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador correlativo de la sesión P2P.
id_jugador_1	INT	NO	FK → jugadores(id)	N/A	Operador iniciador / proponente de la oferta.
id_jugador_2	INT	NO	FK → jugadores(id)	N/A	Operador receptor / contraparte del intercambio.
id_item_j1	INT	SÍ	FK → items(id)	NULL	Ítem ofrecido por el Jugador 1.
id_item_j2	INT	SÍ	FK → items(id)	NULL	Ítem ofrecido por el Jugador 2.
monto_j1	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Efectivo líquido adjunto por Jugador 1.
monto_j2	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Efectivo líquido adjunto por Jugador 2.
confirmacion_j1	BOOLEAN	NO	N/A	FALSE	Aceptación explícita de Jugador 1 (RN010).
confirmacion_j2	BOOLEAN	NO	N/A	FALSE	Aceptación explícita de Jugador 2 (RN010).
estado	VARCHAR(50)	NO	CHECK ('PENDIENTE','ACEPTADO','EN_PROCESO','ESPERANDO_CONFIRMACION_FINAL','COMPLETADO','CANCELADO','EXPIRADO')	'PENDIENTE'	Máquina de estados de la negociación (RN012).
fecha_creacion	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Momento exacto de apertura de la propuesta.
fecha_expiracion	DATETIME	NO	N/A	N/A	Límite temporal antes de anulación por inactividad.

Tabla: transacciones (Cabecera del Libro Mayor Inmutable)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_transaccion	INT	NO	PK, AUTO_INCREMENT	N/A	Número de folio inmutable de la transacción.
id_item_afectado	INT	SÍ	FK → items(id)	NULL	Bien transferido en la operación (si aplica).
id_negociacion	INT	SÍ	FK → negociaciones_tradeos(id)	NULL	Mesa de tradeo que originó el movimiento.
id_jornada	INT	SÍ	FK → jornadas_laborales(id)	NULL	Jornada laboral origen del pago de nómina.
tipo_transaccion	VARCHAR(30)	NO	CHECK ('PAGO_SALARIO','TRADEO_P2P','COMPRA_COMERCIANTE','AJUSTE_ADMIN')	N/A	Clasificación de la operación económica.
estado_transaccion	VARCHAR(20)	NO	CHECK ('COMPLETADA','CANCELADA','REVERTIDA')	'COMPLETADA'	Resultado de auditoría del movimiento (RN003).
monto	DECIMAL(12,2)	NO	CHECK (>= 0)	N/A	Monto nominal bruto movilizado.
fecha_hora	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Marca temporal inmutable de ejecución (RN003).

Tabla: detalles_transacciones (Asientos Contables de Partida Doble)

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	DESCRIPCIÓN Y REGLA VINCULADA
id_detalle	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único del asiento contable.
id_transaccion	INT	NO	FK → transacciones(id)	N/A	Transacción cabecera vinculada.
cuenta_origen	INT	NO	FK → cuentas(id)	N/A	Cuenta bancaria debitada en el movimiento.
cuenta_destino	INT	NO	FK → cuentas(id)	N/A	Cuenta bancaria acreditada en el movimiento.
tipo_movimiento	VARCHAR(10)	NO	CHECK ('DEBITO','CREDITO')	N/A	Sentido contable del asiento por partida doble.
monto_detalle	DECIMAL(12,2)	NO	CHECK (> 0)	N/A	Cuántía monetaria exacta del asiento.
concepto	VARCHAR(80)	SÍ	N/A	NULL	Glosa explicativa (ej. 'COMISION_DRENAJE_SISTEMA').

5. Informe de Mejoras y Proceso de Normalización

5.1 Mejoras Implementadas en la Capa de Datos

- **Garantía de Cuenta Única de Sistema por Servidor:** Se habilitó la cuenta tipo_cuenta = 'SISTEMA' con id_jugador = NULL, permitiendo la inyección lícita de capital salarial y centralizando la recaudación fiscal de drenaje.
- **Atomicidad Transaccional:** Las operaciones complejas de intercambio bilateral y pago laboral se ejecutan en bloques START TRANSACTION ... COMMIT / ROLLBACK con manejadores de error, garantizando que ante cualquier falla los datos se restauren a su estado previo sin inconsistencias.
- **Inclusión de Disparadores e Inmutabilidad:** Protección de las tablas de auditoría contra sentencias UPDATE y DELETE, forzando un libro mayor contable inalterable.
- **Gestión de Ciclo de Vida y Expiración:** Control automático de temporizadores para marcar negociaciones como 'EXPIRADO' cuando superan su ventana de validez.

5.2 Sustentación Formal de Formas Normales (1FN a 4FN / DKNF)

FORMA NORMAL	CRITERIO TEÓRICO APLICADO	RESOLUCIÓN Y GARANTÍA EN ECONOMY RP
1FN (Atomicidad)	Cada celda contiene un único valor escalar indivisible; sin atributos multivaluados ni repetidos.	Se descompuso la lista de participantes en detalles_transacciones (una cuenta origen y una destino por fila) y los bienes en items y negociaciones_tradeos.
2FN (Dependencia Total)	Cumple 1FN y ningún atributo no-clave depende parcialmente de claves compuestas.	Todas las tablas del sistema utilizan claves primarias simples subrogadas (id_jornada, id_cuenta, etc.), anulando matemáticamente dependencias parciales.
3FN (Sin Transitividad)	Cumple 2FN y ningún atributo no-clave depende transitivamente de otro atributo no-clave.	En jornadas_laborales se sustituyó el cálculo dinámico de ganancia por monto_pagado inmutable, evitando depender de cambios futuros en la tarifa base de empleos.
FNBC / BCNF	Todo determinante funcional en el esquema constituye formalmente una superclave.	Se eliminaron dependencias encubiertas soportando todas las relaciones estrictamente sobre claves primarias enteras o índices UNIQUE declarados.
4FN (Multivaluadas)	No existen dependencias multivaluadas no triviales independientes entre sí.	El historial laboral, las cuentas monetarias y los ítems de un jugador residen en tablas independientes vinculadas unívocamente por la FK id_jugador.
DKNF (Dominio/Clave)	Toda restricción es consecuencia directa de la definición de dominios y restricciones de clave.	Todas las reglas de negocio se imponen nativamente mediante restricciones CHECK, NOT NULL, UNIQUE y FOREIGN KEY.

6. Pruebas Estructurales, Hallazgos y Manejo de Errores

Para certificar la robustez del esquema DDL en MySQL 8.0 (InnoDB), se ejecutó una suite exhaustiva de pruebas de estrés estructural e inserción controlada, documentando los códigos de error reales del motor y su correspondiente resolución técnica:

6.1 Bitácora de Errores Encontrados y Mitigados en SGBD

CASO DE ERROR 1: Error de Sintaxis por Palabra Residual (MySQL Error Code: 1064)**Sentencia SQL Probada:**

```
INSERT INTO servidores (nombre, porcentaje_comision, limite_bienes_por_jugador, tiempo_enfriamiento_min) VALUES ('Servidor Prueba', 5.00, 10, 1); servidores
```

Mensaje del Motor SGBD: **Error Code: 1064. You have an error in your SQL syntax; check the manual near 'servidores' at line 1.**

Causa Técnica: Se incluyó accidentalmente la palabra residual 'servidores' al final de la sentencia.

Resolución y Mitigación DDL: Se depuró la instrucción eliminando el token sobrante.

Código Corregido / Verificación:

```
INSERT INTO servidores (nombre, porcentaje_comision, limite_bienes_por_jugador, tiempo_enfriamiento_min) VALUES ('Servidor Prueba', 5.00, 10, 1);
```

CASO DE ERROR 2: Intento de Duplicación en Cuenta Única de Sistema (MySQL Error Code: 1062)**Sentencia SQL Probada:**

```
INSERT INTO cuentas (id_servidor, tipo_cuenta, saldo_inicial, saldo_disponible) VALUES (1, 'SISTEMA', 1000000.00, 1000000.00);
```

Mensaje del Motor SGBD: **Error Code: 1062. Duplicate entry '1' for key 'cuentas.uq_cuenta_sistema_por_servidor'**

Causa Técnica: El servidor 1 ya contaba con una cuenta de Sistema activa. El índice UNIQUE impidió la creación de una segunda cuenta duplicada.

Resolución y Mitigación DDL: CORRECTO: El error valida la regla de gobernanza que prohíbe múltiples tesorerías para un mismo servidor.

Código Corregido / Verificación:

```
SELECT * FROM cuentas WHERE id_servidor = 1 AND tipo_cuenta = 'SISTEMA';
```

CASO DE ERROR 3: Inserción sobre Columna Inexistente (Columna Desconocida) (MySQL Error Code: 1054)**Sentencia SQL Probada:**

```
INSERT INTO jugadores (id_servidor, nombre_usuario, nivel) VALUES (1, 'Usuario Prueba', 1);
```

Mensaje del Motor SGBD: **Error Code: 1054. Unknown column 'nivel' in 'field list'**

Causa Técnica: Se intentó insertar en el campo 'nivel' el cual no existe en la definición DDL normalizada de jugadores.

Resolución y Mitigación DDL: Se sincronizó la consulta de inserción con las columnas oficiales del esquema DDL.

Código Corregido / Verificación:

```
INSERT INTO jugadores (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (1, 'Usuario Prueba', 'user@rp.com', 'hash123');
```

CASO DE ERROR 4: Violación de Integridad Referencial en Jugador (Servidor Inexistente) (MySQL Error Code: 1452)**Sentencia SQL Probada:**

```
INSERT INTO jugadores (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (9999, 'Jugador_Alpha', 'alpha@email.com', 'hash_secret_123');
```

Mensaje del Motor SGBD: **Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('economy_rp`.`jugadores`, CONSTRAINT `fk_jugador_servidor` FOREIGN KEY (`id_servidor`) REFERENCES `servidores` (`id_servidor`))**

Causa Técnica: El motor InnoDB interceptó la FK id_servidor = 9999 que no existe en la tabla padre servidores.

Resolución y Mitigación DDL: CORRECTO: El SGBD bloquea la creación de jugadores huérfanos sin servidor asignado.

Código Corregido / Verificación:

```
INSERT INTO jugadores (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (@id_servidor_real, 'Jugador_Alpha', 'alpha@email.com', 'hash_secret_123');
```

CASO DE ERROR 5: Violación de Integridad Referencial en Cuentas (Jugador Inexistente) (MySQL Error Code: 1452)**Sentencia SQL Probada:**

```
INSERT INTO cuentas (id_jugador, id_servidor, tipo_cuenta, saldo_inicial, saldo_disponible) VALUES (9999, 1, 'PERSONAL', 5000.00, 5000.00);
```

Mensaje del Motor SGBD: **Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('economy_rp`.`cuentas`, CONSTRAINT `fk_cuenta_jugador` FOREIGN KEY (`id_jugador`) REFERENCES `jugadores` (`id_jugador`))**

Causa Técnica: No existe ningún operador registrado con id_jugador = 9999.

Resolución y Mitigación DDL: CORRECTO: Protege la apertura de billeteras sin titular verificado.

Código Corregido / Verificación:

```
INSERT INTO cuentas (id_jugador, id_servidor, tipo_cuenta, saldo_inicial, saldo_disponible) VALUES (@id_jugador_real, 1, 'PERSONAL', 5000.00, 5000.00);
```

CASO DE ERROR 6: Violación de Integridad Referencial en Empleos (Servidor Inexistente) (MySQL Error Code: 1452)**Sentencia SQL Probada:**

```
INSERT INTO empleos (id_servidor, nombre_empleo, tarifa_base) VALUES (9999, 'Minero', 150.00);
```

Mensaje del Motor SGBD: **Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('economy_rp`.`empleos`, CONSTRAINT `fk_empleo_servidor` FOREIGN KEY (`id_servidor`) REFERENCES `servidores` (`id_servidor`))**

Causa Técnica: Se intentó habilitar un empleo sobre un servidor inexistente.

Resolución y Mitigación DDL: CORRECTO: Asegura que todo catálogo laboral esté vinculado a una instancia de juego activa.

Código Corregido / Verificación:

```
INSERT INTO empleos (id_servidor, nombre_empleo, tarifa_base) VALUES (@id_servidor_real, 'Minero', 150.00);
```

6.2 Suite de Pruebas de Validación de Integridad y Lógica Transaccional

A continuación se documentan las pruebas avanzadas de validación de reglas de negocio y manejo de excepciones mediante bloques transaccionales y disparadores:

PRUEBA 8: Horas Laborales Inválidas (Negativas o Cero)

Objetivo de la Prueba: Validar que el motor rechace jornadas laborales con horas menores o iguales a cero (RN001).

Sentencia SQL Ejecutada:

```
INSERT INTO jornadas_laborales (id_jugador, id_empleo, horas_trabajadas) VALUES (1, 1, -10.0);
```

Resultado Esperado: Error de validación y aborto inmediato de la inserción sin alterar saldos.

Diagnóstico de Riesgo Mitigado: Si no se controla, un valor negativo generaría montos pagados negativos, drenando saldo indebido al jugador.

Restricción / Código DDL de Blindaje:

```
CONSTRAINT chk_horas_positivas CHECK (horas_trabajadas > 0) -- y validación en Trigger con SIGNAL SQLSTATE '45000' 'Las horas trabajadas deben ser mayores a cero';
```

PRUEBA 9: Intento de Robo / Oferta de Ítem Ajeno en Tradeo

Objetivo de la Prueba: Verificar que un operador solo pueda incluir en una propuesta de intercambio activos que le pertenezcan legítimamente (RN002).

Sentencia SQL Ejecutada:

```
CALL sp_abrir_negociacion(@jugador_A, @jugador_B, @item_de_C, NULL, 0, 0, 10);
```

Resultado Esperado: El procedimiento intercepta la no coincidencia de titularidad y aborta con error 45000.

Diagnóstico de Riesgo Mitigado: Garantiza que ningún bien pueda ser negociado por un tercero no autorizado.

Restricción / Código DDL de Blindaje:

```
IF NOT EXISTS (SELECT 1 FROM items WHERE id_item = p_id_item_j1 AND id_jugador = p_id_jugador_1) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El item ofrecido no le pertenece al proponente'; END IF;
```

PRUEBA 10: Confirmación de Negociación Fuera de Tiempo (Expirada)

Objetivo de la Prueba: Comprobar que una sesión de tradeo vencida no pueda ser confirmada ni ejecute movimientos de fondos (RN010, RN012).

Sentencia SQL Ejecutada:

```
CALL sp_confirmar_tradeo(@id_negociacion_expirada, @jugador_A);
```

Resultado Esperado: Error 'La negociación ya expiró' y transición del estado a 'EXPIRADO'.

Diagnóstico de Riesgo Mitigado: Evita que se ejecuten tratos obsoletos cuyos términos ya no son válidos para ambas partes.

Restricción / Código DDL de Blindaje:

```
IF v_fecha_exp < NOW() THEN UPDATE negociaciones_tradeos SET estado = 'EXPIRADO' WHERE id_negociacion = p_id; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La negociacion ya expiro'; END IF;
```

PRUEBA 11: Exceso de Límite de Inventario por Tradeo

Objetivo de la Prueba: Verificar que el límite de propiedades por servidor se respete al recibir bienes vía tradeo P2P (RN007).

Sentencia SQL Ejecutada:

```
CALL sp_confirmar_tradeo(@negociacion, @jugador_receptor_lleno);
```

Resultado Esperado: El procedimiento calcula el balance neto de ítems; si supera el límite del servidor, cancela la negociación y arroja error 45000.

Diagnóstico de Riesgo Mitigado: Evita el acaparamiento de activos en una sola cuenta mediante transferencias entre jugadores.

Restricción / Código DDL de Blindaje:

```
IF (v_cant_actual - 1 + 1) > v_limite_servidor THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El tradeo excede el limite de bienes permitido'; END IF;
```

PRUEBA 12: Intento de Tradeo Consigo Mismo

Objetivo de la Prueba: Comprobar que un operador no pueda abrir una sala de intercambio con su propia cuenta (RN012).

Sentencia SQL Ejecutada:

```
INSERT INTO negociaciones_tradeos (id_jugador_1, id_jugador_2, ...) VALUES (1, 1, ...);
```

Resultado Esperado: Violación de restricción CHECK y bloqueo inmediato en el motor relacional.

Diagnóstico de Riesgo Mitigado: Evita el auto-bloqueo de sesiones y el colapso de la máquina de estados de doble confirmación.

Restricción / Código DDL de Blindaje:

```
CONSTRAINT chk_negociacion_distintos CHECK (id_jugador_1 <> id_jugador_2)
```

7. Defensa Técnica y Sustentación del Esquema

La arquitectura del esquema DDL de Economy RP en MySQL 8.0 (InnoDB) no es un repositorio pasivo, sino un motor transaccional gobernado por reglas estrictas de integridad relacional. A nivel de defensa técnica, se sustenta el comportamiento del sistema respondiendo a las preguntas fundamentales de flujo de información:

1. ¿Por qué el modelo restringe el traspaso de bienes en ciertas condiciones?

El modelo bloquea de forma intransigente la transferencia de cualquier ítem que registre gravámenes (`tiene_deuda = TRUE`, RN004) o cuyo titular no coincida con el operador autenticado (`items.id_jugador != emisor`, RN002). Asimismo, el motor rechaza la transacción si el receptor ya alcanzó su cuota máxima de inventario (RN007) o si no dispone de liquidez suficiente para cubrir el importe acordado más el 5% de comisión tributaria (RN006). Estas restricciones protegen a la comunidad de estafas, venta fraudulenta de vehículos embargados y sobregiros contables.

2. ¿Por qué el modelo permite la emisión de dinero sólo a través de canales específicos?

El esquema prohíbe la creación de dinero 'de la nada' mediante inserciones directas en cuentas personales. Todo flujo de entrada a la economía debe ser respaldado por una jornada laboral completada (jornadas_laborales) y registrado como un débito formal a la cuenta oficial de Tesorería del SISTEMA (RN005). Esto asegura que la masa monetaria en circulación sea 100% auditable mediante la suma simétrica de débitos y créditos en detalles_transacciones.

3. ¿Por qué se implementó Inmutabilidad Contable por Partida Doble?

En un entorno financiero virtual, la modificación arbitraria de saldos mediante sentencias UPDATE destruye la trazabilidad histórica. Al estructurar transacciones y detalles_transacciones bajo un patrón de sólo adición (*append-only*), cualquier ajuste administrativo debe realizarse mediante un nuevo asiento compensatorio, protegiendo la fe pública y la confianza de los participantes.

4. Garantía de Propiedades ACID en el Motor Relacional:

- **Atomicidad (A):** Todas las operaciones de intercambio P2P y liquidación laboral se ejecutan en transacciones explícitas con manejadores EXIT HANDLER FOR SQLEXCEPTION ROLLBACK, garantizando que todo se aplique en su totalidad o se revierta por completo.
- **Consistencia (C):** El cumplimiento riguroso de 1FN a 4FN, las llaves foráneas y las restricciones CHECK aseguran que la base de datos nunca quede en un estado inválido.
- **Aislamiento (I):** Las consultas de verificación emplean bloqueos pesimistas (SELECT ... FOR UPDATE), evitando condiciones de carrera en operaciones concurrentes.
- **Durabilidad (D):** Los registros persistidos en MySQL con el motor transaccional InnoDB garantizan la recuperación total ante fallos de hardware o caídas de red.

8. Anexo: Modificaciones y Resoluciones tras Retroalimentación Sincrónica

A continuación se sintetizan formalmente las mejoras aplicadas a partir de las observaciones del docente asesor en la sesión sincrónica:

OBSERVACIÓN DOCENTE	RIESGO RELACIONAL DIAGNOSTICADO	SOLUCIÓN DDL IMPLEMENTADA
Punto 1: Estructura de Ítems y Posesión Rígida	FK id_jugador rígida (1:N) impedía que los bienes pertenecieran a concesionarios o quedaran bloqueados en tradeo sin perder al dueño legítimo.	Se modularizó con id_jugador INT NULL y se incorporaron estado_custodia CHECK ('PERSONAL','EMBARGADO','EN_TRADE') y tipo_propietario CHECK ('JUGADOR','CONCESIONARIO','SISTEMA').
Punto 2: Cardinalidad de Cuentas Financieras	Restricción UNIQUE en id_jugador forzaba relación 1:1, impidiendo cuentas múltiples (ahorro, comercial, corporativa).	Se eliminó la restricción UNIQUE en cuentas.id_jugador, habilitando formalmente cardinalidad 1:N clasificada mediante tipo_cuenta CHECK (...).
Punto 3: Atomicidad y Simetría Contable	Riesgo de descuadre contable si la simetría débito/crédito dependía únicamente de inserciones externas.	Se blindó mediante bloques transaccionales atómicos ACID con bloqueos pesimistas en InnoDB, asegurando SUM(Débitos) = SUM(Créditos) con ROLLBACK automático ante cualquier falla.
Punto 4: Estandarización de Nomenclatura	Uso de prefijo T_ y singularidades inconsistentes.	Se normalizó el DDL completo eliminando el prefijo T_ y estandarizando nombres de tablas en plural (servidores, jugadores, etc.) en snake_case minúscula.

Miguel Ángel Cardona Agudelo
Desarrollo DB & DDL

Luisa María López Orrego
Normalización & Calidad

Jorge Luis Ordoñez Ávila
Pruebas & Auditoría